

Decorator Design Pattern

Coffee Shop Ordering System

Agenda

- 1 Motivation
- 2 Naive Solution
- 3 Problems
- 4 Decorator Pattern
- 5 Class Diagram
- 6 Implementation
- 7 Advantages and Disadvantages
- 8 Real-world Applications
- 9 Mini Quiz

1. A Real-world Problem

Imagine your team is building an order management software for a coffee shop chain (e.g., The Coffee House).

Available coffee:

- Dark Roast
- House Blend
- Espresso

Customers rarely order plain coffee.

They often request additional condiments:

- Milk
- Sugar
- Caramel

Each topping changes

- Price
- Description

2. Naive Solution - Inheritance

The first idea is using inheritance.

Example:

- Beverage
- DarkRoast
- DarkRoastWithMilk
- DarkRoastWithMilkAndSugar
- DarkRoastWithMilkAndSugarAndWhip
- EspressoWithWhip
- HouseBlendWithMilk
- ...

2. Naive Solution - Inheritance

The first idea is using inheritance.

Example:

- Beverage
- DarkRoast
- DarkRoastWithMilk
- DarkRoastWithMilkAndSugar
- DarkRoastWithMilkAndSugarAndWhip
- EspressoWithWhip
- HouseBlendWithMilk
- ...

Problem

Every combination requires a new class.

Naive Solution - Boolean Flags

Another approach is storing every topping inside the Beverage class.

```
1 class Beverage {  
2  
3 private:  
4  
5     bool hasMilk;  
6     bool hasSugar;  
7     bool hasWhip;  
8     bool hasCaramel;  
9  
10 public:  
11     ...  
12 };
```

Naive Solution - Boolean Flags

Another approach is storing every topping inside the Beverage class.

```
1 class Beverage {  
2  
3 private:  
4  
5     bool hasMilk;  
6     bool hasSugar;  
7     bool hasWhip;  
8     bool hasCaramel;  
9  
10 public:  
11     ...  
12 };
```

Everything is controlled by boolean variables.

3. Problems with the Naive Solution

Class Explosion

4 coffee types $\times$ many toppings = dozens (or hundreds) of subclasses.

3. Problems with the Naive Solution

Class Explosion

4 coffee types $\times$ many toppings = dozens (or hundreds) of subclasses.

Hard Maintenance

Changing the price of milk requires modifying many classes.

3. Problems with the Naive Solution

Class Explosion

4 coffee types $\times$ many toppings = dozens (or hundreds) of subclasses.

Hard Maintenance

Changing the price of milk requires modifying many classes.

Violates Open/Closed Principle

Adding a new topping (e.g. Boba) requires modifying Beverage.

3. Problems with the Naive Solution

Class Explosion

4 coffee types $\times$ many toppings = dozens (or hundreds) of subclasses.

Hard Maintenance

Changing the price of milk requires modifying many classes.

Violates Open/Closed Principle

Adding a new topping (e.g. Boba) requires modifying Beverage.

No Runtime Flexibility

Cannot naturally support

- Double Milk
- Triple Mocha

4. Decorator Pattern

Decorator is a **Structural Design Pattern**.

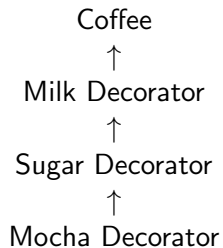
Instead of inheritance,
we dynamically attach new behaviors to an object.

Core philosophy:

Composition over Inheritance

Core Concepts

Decorator uses wrapper objects.



5. General Structure of Decorator Pattern

Main Components

- **Component:** Common interface for all objects.
- **ConcreteComponent:** The original object.
- **Decorator:** Abstract wrapper containing a Component.
- **ConcreteDecorator:** Adds new behavior while preserving the interface.

Flow:

Component $\leftarrow$ *Decorator* $\leftarrow$ *Decorator* $\leftarrow$ *Decorator*

Every decorator is still recognized as a Component.

Coffee UML (Mermaid)

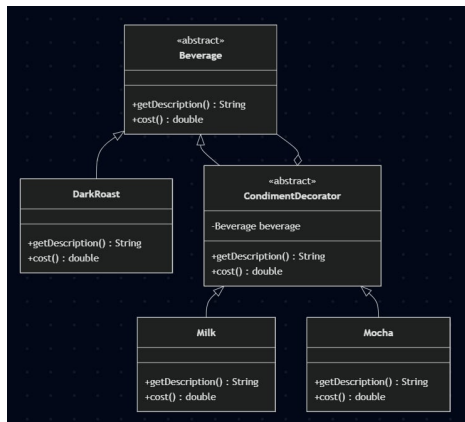


Figure: Coffee UML diagram

7. Component

```
1 class Beverage {  
2 public:  
3     virtual string getDescription() const = 0;  
4     virtual double cost() const = 0;  
5     virtual ~Beverage() = default;  
6 };
```


7. Component

```
1 class Beverage {  
2 public:  
3     virtual string getDescription() const = 0;  
4     virtual double cost() const = 0;  
5     virtual ~Beverage() = default;  
6 };
```

Role

Defines the common interface. Every coffee and every topping must inherit from Beverage.

Concrete Component

```
1 class DarkRoast : public Beverage {  
2 public:  
3     string getDescription() const override {  
4         return "Dark_Roast_Coffee";  
5     }  
6  
7     double cost() const override {  
8         return 30.0;  
9     }  
10 };
```

Concrete Component

```
1 class DarkRoast : public Beverage {  
2 public:  
3     string getDescription() const override {  
4         return "Dark_Roast_Coffee";  
5     }  
6  
7     double cost() const override {  
8         return 30.0;  
9     }  
10 };
```

DarkRoast is the original object before decoration.

Base Decorator

```
1 class CondimentDecorator : public Beverage {  
2 protected:  
3     Beverage* beverage;  
4 public:  
5     CondimentDecorator(Beverage* bev) : beverage(bev) {}  
6  
7     virtual ~CondimentDecorator() {  
8         delete beverage;  
9     }  
10 };
```

Base Decorator

```
1 class CondimentDecorator : public Beverage {  
2 protected:  
3     Beverage* beverage;  
4 public:  
5     CondimentDecorator(Beverage* bev) : beverage(bev) {}  
6  
7     virtual ~CondimentDecorator() {  
8         delete beverage;  
9     }  
10 };
```

The decorator wraps another Beverage object.
Every decorator stores a pointer to another Beverage.

Concrete Decorator: Milk

```
1 class Milk : public CondimentDecorator {  
2 public:  
3     Milk(Beverage* bev) : CondimentDecorator(bev) {}  
4  
5     string getDescription() const override {  
6         return beverage->getDescription() + ", Milk";  
7     }  
8  
9     double cost() const override {  
10        return beverage->cost() + 5.0;  
11    }  
12};
```

Concrete Decorator: Milk

```
1 class Milk : public CondimentDecorator {  
2 public:  
3     Milk(Beverage* bev) : CondimentDecorator(bev) {}  
4  
5     string getDescription() const override {  
6         return beverage->getDescription() + ", Milk";  
7     }  
8  
9     double cost() const override {  
10        return beverage->cost() + 5.0;  
11    }  
12};
```

Concrete Decorator: Mocha

```
1 class Mocha : public CondimentDecorator {
2 public:
3     Mocha(Beverage* bev) : CondimentDecorator(bev) {}
4     string getDescription() const override { return beverage->
        getDescription() + ", Mocha";
5     }
6
7     double cost() const override {
8         return beverage->cost() + 8.0;
9     }
10 };
```


Concrete Decorator: Mocha

```
1 class Mocha : public CondimentDecorator {  
2 public:  
3     Mocha(Beverage* bev) : CondimentDecorator(bev) {}  
4     string getDescription() const override { return beverage->  
        getDescription() + ", Mocha";  
5     }  
6  
7     double cost() const override {  
8         return beverage->cost() + 8.0;  
9     }  
10 };
```

Every new decorator extends the previous Beverage.
No existing class needs modification.

Client Code

```
1 int main() {  
2     Beverage* myDrink = new DarkRoast();  
3  
4     myDrink = new Milk(myDrink);  
5     myDrink = new Mocha(myDrink);  
6  
7     cout << myDrink->getDescription() << endl;  
8     cout << myDrink->cost();  
9     delete myDrink;  
10 }
```

How Decoration Works

The object is wrapped layer by layer.

DarkRoast



Milk(DarkRoast)



Mocha(Milk(DarkRoast))

How Decoration Works

The object is wrapped layer by layer.

DarkRoast



Milk(DarkRoast)



Mocha(Milk(DarkRoast))

Every wrapper is still a Beverage.

Execution Flow

Calling

myDrink - $\rightarrow$ *cost()*

starts from the outermost decorator.

Execution order:

Mocha $\rightarrow$ *Milk* $\rightarrow$ *DarkRoast*

Then the values are returned back.

$$8 + 5 + 30 = 43$$

Program Output

Description

Dark Roast Coffee, Milk, Mocha

Total Cost

43k VND

Advantages

Runtime Flexibility

Features can be added dynamically while the program is running.

Advantages

Runtime Flexibility

Features can be added dynamically while the program is running.

Unlimited Combinations

Supports any topping combination without creating new subclasses.

Advantages

Runtime Flexibility

Features can be added dynamically while the program is running.

Unlimited Combinations

Supports any topping combination without creating new subclasses.

Reusable Components

Each decorator can be reused independently.

Advantages

Runtime Flexibility

Features can be added dynamically while the program is running.

Unlimited Combinations

Supports any topping combination without creating new subclasses.

Reusable Components

Each decorator can be reused independently.

Follows SOLID

- Single Responsibility Principle
- Open/Closed Principle

Disadvantages

Many Small Classes

Each feature requires its own decorator class.

Disadvantages

Many Small Classes

Each feature requires its own decorator class.

Harder Debugging

Many wrapper layers produce long call stacks.

Disadvantages

Many Small Classes

Each feature requires its own decorator class.

Harder Debugging

Many wrapper layers produce long call stacks.

Complex Initialization

Nested construction may become difficult.

Example:

```
new Mocha(new Milk(new Mocha(new DarkRoast()))))
```

9. Java I/O Streams

Decorator is widely used in the Java Standard Library.

Example:

```
new BufferedInputStream( new FileInputStream("data.txt"))
```

9. Java I/O Streams

Decorator is widely used in the Java Standard Library.

Example:

```
new BufferedInputStream( new FileInputStream("data.txt"))
```

Idea

`FileInputStream` provides basic file reading.

`BufferedInputStream` wraps it and adds buffering for higher performance.

The original object is not modified.

React Higher-Order Components (HOC)

A Higher-Order Component (HOC) wraps another component.

Example:

withAuth(ProfilePage)

React Higher-Order Components (HOC)

A Higher-Order Component (HOC) wraps another component.

Example:

withAuth(ProfilePage)

The wrapper may add:

- Authentication
- Logging
- Theme
- Error Handling

without changing the original component.

Backend Middleware

Frameworks such as Express.js use the same idea.

Example

Auth → *Logger* → *Controller*

Each middleware wraps the next one.

Backend Middleware

Frameworks such as Express.js use the same idea.

Example

Auth → *Logger* → *Controller*

Each middleware wraps the next one.

Typical responsibilities:

- Authentication
- Authorization
- Logging
- Request Validation

Python Decorators

Python decorators are direct implementations of the Decorator idea.

Example:

```
1 @login_required
2 def dashboard():
3     ...
```

Python Decorators

Python decorators are direct implementations of the Decorator idea.

Example:

```
1 @login_required
2 def dashboard():
3     ...
```

The function is wrapped before execution.

Additional behavior can be added without modifying the original function.

Flutter Widgets

Flutter also encourages composition.

Example

```
Container(Padding(Text(" Hello" )))
```

Flutter Widgets

Flutter also encourages composition.

Example

```
Container(Padding(Text(" Hello" )))
```

Each widget wraps another widget.

Instead of inheritance,

Flutter builds UI through composition.

Question 1

Decorator belongs to which Design Pattern category?

- A. Creational
- B. Structural
- C. Behavioral
- D. Architectural

Question 1

Decorator belongs to which Design Pattern category?

- A. Creational
- B. Structural
- C. Behavioral
- D. Architectural

Answer

B. Structural

Prize: Sweet Candy

Question 2

Which design principle does Decorator emphasize?

- A. Inheritance over Composition
- B. Composition over Inheritance
- C. Tight Coupling
- D. Procedural Programming

Question 2

Which design principle does Decorator emphasize?

- A. Inheritance over Composition
- B. Composition over Inheritance
- C. Tight Coupling
- D. Procedural Programming

Answer

B. Composition over Inheritance

Prize: Cake

Question 3

What is the main purpose of Decorator?

- A. Clone an object
- B. Restrict object creation
- C. Add new behavior at Runtime
- D. Simplify a subsystem

Question 3

What is the main purpose of Decorator?

- A. Clone an object
- B. Restrict object creation
- C. Add new behavior at Runtime
- D. Simplify a subsystem

Answer

C. Add new behavior at Runtime

Prize: Virtual Hug

Question 4

Which is a disadvantage of Decorator?

- A. Violates OCP
- B. Causes Class Explosion
- C. Long call stack and many small classes
- D. Cannot work in OOP languages

Question 4

Which is a disadvantage of Decorator?

- A. Violates OCP
- B. Causes Class Explosion
- C. Long call stack and many small classes
- D. Cannot work in OOP languages

Answer

C

Prize: Chocolate

Question 5

Which concept is closest to Decorator?

- A. Database Index
- B. REST API
- C. CSS Styling
- D. Middleware / HOC

Question 5

Which concept is closest to Decorator?

- A. Database Index
- B. REST API
- C. CSS Styling
- D. Middleware / HOC

Answer

D

Prize: Flying Kiss

Question 6

How do we implement Double Milk?

- A. Create DoubleMilkDecorator
- B. Add count = 2
- C. Wrap Milk twice
- D. Copy the code twice

Question 6

How do we implement Double Milk?

- A. Create DoubleMilkDecorator
- B. Add count = 2
- C. Wrap Milk twice
- D. Copy the code twice

Answer

C

Prize: Bubble Milk Tea

Summary

Today we learned

- The motivation behind Decorator

Summary

Today we learned

- The motivation behind Decorator
- Problems with inheritance

Summary

Today we learned

- The motivation behind Decorator
- Problems with inheritance
- Composition over Inheritance

Summary

Today we learned

- The motivation behind Decorator
- Problems with inheritance
- Composition over Inheritance
- UML Structure

Summary

Today we learned

- The motivation behind Decorator
- Problems with inheritance
- Composition over Inheritance
- UML Structure
- C++ Implementation

Summary

Today we learned

- The motivation behind Decorator
- Problems with inheritance
- Composition over Inheritance
- UML Structure
- C++ Implementation
- Advantages and Disadvantages

Summary

Today we learned

- The motivation behind Decorator
- Problems with inheritance
- Composition over Inheritance
- UML Structure
- C++ Implementation
- Advantages and Disadvantages
- Practical Applications

Thank You!

Questions?